

Jenkins CI/CD Lab — Practice Questions

ISWE406L — Agile Development Process and DevOps | Windows Agent

Question 1

Create a Jenkins pipeline on a Windows agent with three stages: **Checkout** from your GitHub repository, **Install Dependencies** from a `requirements.txt` file, and **Run Unit Tests** with `pytest`. Write an `app.py` containing `multiply(a, b)` and `divide(a, b)`, and a `test_app.py` with one test function for each. Add a post block that prints one message on success and another on failure. Run the build once so it passes. Then break `multiply` so it returns a wrong value, push the change, rebuild, and show which post message appears.

Question 2

Build a pipeline with the stages **Checkout**, **Install Dependencies**, and **Run Unit Tests (verbose)**. Write an `app.py` with `find_min(numbers)` and `count_odds(numbers)`. In `test_app.py`, use `@pytest.mark.parametrize` to test each function against three input cases, and run `pytest` with the `-v` flag. Show from the Console Output how many tests ran and explain why that number differs from the number of test functions. Then add one case with a wrong expected value and show that only that case fails.

Question 3

Design a pipeline with the stages **Checkout**, **Build** (a compile check of `app.py` using `py_compile`), and **Deploy**. Define `APP_NAME` and `APP_VERSION` as custom variables in an environment block. The Deploy stage must pause with an input step that asks "Approve deployment of <APP_NAME> version <APP_VERSION>?" and has an OK button labelled "Release". It should then run `app.py`. Demonstrate both outcomes, clicking **Release** in one build and **Abort** in another, and explain the result of each.

Question 4

Create a pipeline with the stages **Checkout**, **Show Build Info**, and **Run Linter**. The Show Build Info stage should print `BUILD_NUMBER`, `JOB_NAME`, and `WORKSPACE`, and the Run Linter stage should run `flake8` on an `app.py` containing a `greet(name)` function. Run the build twice and compare which values change and which stay the same. Then add an unused import to `app.py`, rebuild, and show the linter failure.

Question 5

Write a parameterized pipeline that uses a choice parameter named `ENVIRONMENT` with the options `dev`, `staging`, and `prod`, and a booleanParam named `RUN_EXTRA_CHECK`. The stages should be **Checkout**, **Show Parameter** (which echoes the selected environment), and **Extra Check**. The Extra Check stage runs only when `RUN_EXTRA_CHECK` is true, which you should control with a `when` directive. Explain why the first build does not show the **Build with Parameters** option. Then run builds with the checkbox checked and unchecked and compare the pipeline views.

Question 6

Create a pipeline with the stages **Checkout**, **Parallel Checks**, and **Archive Reports**. Inside **Parallel Checks**, run `frontend_check.py` and `backend_check.py` side by side. Each script should sleep for 4 seconds and write its own result file (`frontend_report.txt` and `backend_report.txt`). Archive both files with `archiveArtifacts`. Estimate the total time of the parallel stage compared with running the scripts one after another. Then show that an older build's artifacts are still available after a newer build runs.

Question 7

Build a pipeline with the stages **Checkout**, **Build**, and **Send Notification**. The **Build** stage should compile `app.py`, wait 15 seconds, and then pass `milestone(1)`. The **Send Notification** stage should use the `mail` step, or the `echo` workaround if SMTP is not configured, with the recipient, a subject containing `JOB_NAME` and `BUILD_NUMBER`, and a body with the build URL. Click **Build Now** twice in quick succession and explain what happens to the older build and whether it sends a notification. Finally, introduce a syntax error in `app.py` and explain why no email is sent.